Text-attributed Graph Augmented Large Language Models for Question Answering

Zhongkun Zhu[1], Junwei Zhang[2], Dongjin Yu[1(✉)], and Xiaolin Li[2]

[1] Hangzhou Dianzi University, Hangzhou, China
{zhuzhongkun,yudj}@hdu.edu.cn
[2] Hangzhou Institute of Medicine, Chinese Academy of Sciences, Hangzhou, China
zhangjunwei@him.cas.cn, xiaolinli@ieee.org

Abstract. In recent years, an increasing number of studies have explored the use of text-attributed graphs to enhance the question-answering capabilities of Large Language Models (LLMs). However, these studies often struggle to effectively integrate structured knowledge and textual information into LLMs. To this end, this paper proposes a novel method to seamlessly integrate text-attributed graphs with pre-trained LLMs. Specifically, the method includes three core components: a graph encoder for capturing graph structural semantics, a graph-text alignment adapter for aligning the semantic spaces of text and graph representations, and a cross-attention module for dynamically fusing graph information into LLM embeddings. Extensive experiments on tasks of commonsense reasoning, scene graph question answering, and multi-hop knowledge graph question answering demonstrate that the method significantly improves LLM's comprehension of text-attributed graph information, outperforming existing baselines.

Keywords: Large Language Models · Question answering · Text-attributed graphs

1 Introduction

Recent advancements in Large Language Models (LLMs), such as GPT-4 [12] and LLaMA [17], have demonstrated powerful capabilities in text generation, logical reasoning, and question answering [11]. However, LLMs are primarily designed for unstructured text, leaving their potential in handling structured data underexplored. In many practical scenarios, textual data is often combined with structured information which is typically represented in the form of graphs [7]. For example, Knowledge Graphs (KGs) represent a typical graph structure, where nodes denote entities (e.g., people, locations, events) and edges represent relationships (e.g., "born in", "belongs to"), both typically extracted from textual data [6].

Therefore, enabling LLMs to process textual information and structural features within text-attributed graphs is gradually emerging as a significant research focus [2]. Existing approaches typically employ Graph Neural Networks

(GNNs) to encode structural information and obtain graph embeddings through pooling operations, which are then injected into LLMs as prefixes [3]. However, such approaches to graph embeddings may fail to fully capture the complex semantic information and structural features within graphs [15]. Furthermore, some researchers directly integrate graph information into LLMs, which introduces substantial redundancy and causes noise interference [16]. Thus, guiding LLMs to focus on query-relevant information while effectively filtering out noise remains a critical challenge.

To address the aforementioned challenges, this paper proposes a novel method aimed at enhancing the ability of LLMs to comprehend key textual and structural information within text-attributed graphs. The proposed method comprises three core components: (1) a graph encoder that captures structural features, (2) a graph-text alignment adapter that maps graph representations into the LLM semantic space, and (3) a cross-attention module that enables the query text after multi-layer semantic modeling to effectively focus on the relevant node embeddings. To comprehensively evaluate our proposed method, we conduct experiments on the GraphQA benchmark [3], which includes three datasets spanning diverse domains: ExplaGraphs (commonsense reasoning), SceneGraphs (scene graph question answering), and WebQSP (multi-hop knowledge graph question answering). The experimental results demonstrate that our method outperforms the best baselines on these tasks by +6.21%, +0.64%, and +4.21%, respectively. To summarize, our contributions are as follows:

- We propose to incorporate a cross attention module exclusively at the last layer of the LLM, demonstrating that the cross attention mechanism remains effective for fusing high-dimensional semantic embeddings.
- We introduce a new method for injecting graph-structural information into the LLM, enabling the LLM to more effectively capture graph features and thereby reduce information loss.
- The superiority of the proposed method is demonstrated on three datasets from distinct domains, with ablation study, model design comparison, and parameter sensitivity analyses further validating its effectiveness.

2 Related Work

2.1 Large Language Models and Parameter-Efficient Fine-Tuning

Based on the Transformer architecture [18] and pre-trained on diverse large-scale text corpora, LLMs have demonstrated remarkable performance across a wide range of natural language processing (NLP) tasks. Prominent models like GPT-4 [12] and LLaMA [17] demonstrate that scaling model parameters and dataset size significantly enhances performance in language understanding, generation, and related tasks. However, the growing scale of LLMs presents significant challenges for downstream fine-tuning, particularly in low-resource or task-specific scenarios where full-parameter updates are computationally intensive and susceptible to overfitting [10]. To tackle these challenges, Parameter-Efficient Fine-Tuning (PEFT) methods have emerged, enabling efficient adaptation by freezing

pretrained weights and updating only a small subset of parameters. Adapter-based approaches [4, 14] insert small, trainable modules into each Transformer layer, while prompt-based methods [8, 9] learn task-specific prompts without altering model parameters. Low-Rank Adaptation (LoRA) [5] and its variants have gained popularity due to their effectiveness and low memory overhead by injecting low-rank updates into attention and feedforward layers.

2.2 Integrating Graph Information into LLMs for Question Answering

Early approaches directly feed triplet facts relevant to the question into the LLMs [1] or append instructions after textual descriptions of the graph [19], such as "Let's construct a graph with the nodes and edges first", hoping that LLMs can understand the graph structure. However, in the absence of structure-aware mechanisms, LLMs often struggle to comprehend structural information embedded in serialized graphs and are prone to generate irrelevant or noisy outputs. To more effectively capture graph structures, subsequent studies have incorporated graph neural networks (GNNs) to model subgraphs and employed specialized fusion strategies to enable LLMs to understand graph representations. For instance, GreaseLM [20] models text and graphs using language models and GNNs, respectively, and then performs modality fusion via interactions between special token and node. GNP [16] uses a GNN to capture structural information and designs a cross-modality pooling module to obtain the most relevant node embeddings as graph neural prompts.

3 Method

Our method integrates textual and structural information from text-attributed graphs into LLMs to enhance question-answering performance. As shown in Figure 1, it consists of three key components: (1) a graph encoder, (2) a graph-text alignment adapter, and (3) a cross-attention module, which are detailed in the following sections.

3.1 Preliminary

Text-attributed Graphs. A text-attributed graph [3] is a graph where nodes and edges are enriched with textual attributes. Formally, it can be defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_{\mathcal{V}}, \mathcal{T}_{\mathcal{E}})$, where $\mathcal{V}$ and $\mathcal{E}$ denote the node set and the edge set, respectively. Additionally, $\mathcal{T}_{\mathcal{V}} = \{x_v\}_{v \in \mathcal{V}}$ and $\mathcal{T}_{\mathcal{E}} = \{x_e\}_{e \in \mathcal{E}}$ signify the collections of textual attributes associated with the nodes and edges, respectively.

Task Description. Text-attributed Graphs Question Answering. Given a natural language question Q and its corresponding subgraph $\mathcal{G}_{sub}$ with text-attributed nodes and edges, the task is to design a machine learning model M to answer the question related to the subgraph by integrating the rich contextual information of the subgraph.

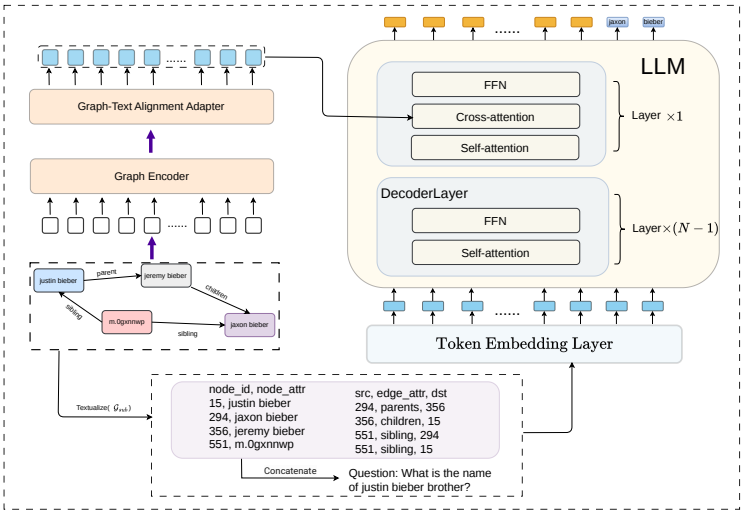

Fig. 1. Overview. Given a question and its corresponding subgraph, we first feed the question concatenated with the textualized subgraph into the LLM for language modeling through $(N-1)$ self-attention layers. Simultaneously, a graph encoder captures structural features, which are then aligned with the LLM's semantic space via a graph-text alignment adapter. Finally, we employ a cross-attention module where text representations serve as the query, and node representations act as the key and value. This fusion integrates structural information to enhance the LLM's reasoning capability.

3.2 Graph Encoder

Although LLM excels at processing natural language text, their ability to comprehend graph-structured data remains limited. Directly serializing subgraphs into text inputs can lead to the loss of topological information (e.g., node paths and adjacency relationships), whereas the answers may not be directly related to the questions. In such cases, modeling the complex structural features between entities can help enhance the reasoning capabilities of LLM. To address this, we introduce a Graph Neural Network (GNN) as the graph encoder to dynamically aggregate neighborhood information and model multi-hop dependencies, thereby enhancing the representation capabilities of entities. Specifically, we first utilize a pre-trained language model to generate initial embeddings for nodes and edges. Next, a standard graph attention network is employed as our graph encoder for the subgraph $\mathcal{G}_{sub}$:

$$H_g = \text{GNN}(\mathcal{G}_{sub}) \in \mathbb{R}^{n \times d_g} \tag{1}$$

where H_g denotes the structure-aware node representations generated by the GNN over the subgraph $\mathcal{G}_{sub}$, n denotes the number of nodes in $\mathcal{G}_{sub}$, and d_g is the dimension of the graph encoder.

3.3 Graph-Text Alignment Adapter

After processing by the graph encoder, we obtain node embeddings that encode structural information. However, these node embeddings reside in a different representation space from the LLM's token embeddings, resulting in semantic misalignment that hinders the LLM's ability to effectively integrate graph-structured information. To address this limitation, we introduce a Graph-Text Alignment Adapter that bridges the modality gap between graph and text representations, enabling effective cross-modal integration. Specifically, we implement this adapter as a Multi-Layer Perceptron (MLP), leveraging its non-linear mapping capability to project the node embeddings into the same semantic space as the LLM's token embeddings. The mapping process is formulated as follows:

$$H'_g = \text{MLP}(H_g) \in \mathbb{R}^{n \times d_l} \tag{2}$$

where d_l is the dimension of the LLM's hidden embedding.

3.4 Graph-Text Integration with Cross-Attention

The integration strategy of graph information plays a decisive role in the final performance of LLM. Simply serializing the graph would introduce excessive noise, and using only a GNN to learn global graph embeddings may result in significant information loss. To address this, we propose a novel fusion strategy that fully leverages the advantages of both graph structure and textual information. Specifically, we first adopt the textualization method introduced in G-Retriever [3] to transform the subgraph $\mathcal{G}_{sub}$ into a textual sequence and concatenate it with the question. Then, we get the input embeddings $H_t^0 \in \mathbb{R}^{L \times d_l}$ by using the token embedding layer of the LLM, where L is the length of the input sequence. The input embeddings are passed through the $(N-1)$ decoder layers of the LLM to model intra-textual dependencies and obtain contextualized text representations:

$$\begin{gathered} H_t^{(i)} = \text{DecoderLayer}^{(i)}(H_t^{(i-1)}) \in \mathbb{R}^{L \times d_l}, \\ \text{for } i = 1, \ldots, N-1 \end{gathered} \tag{3}$$

where N is the number of the decoder layers in the LLM. Next, at the final decoder layer of the LLM, we incorporate a cross-attention module to enable fine-grained interaction between textual and structural information. The text embeddings from the $(N-1)$-th layer are first processed by the standard self-attention operation. The resulting representations then participate in a multi-head cross-attention computation, where they serve as query (Q), and the node embeddings are used as key (K) and value (V). The output of the cross-attention is subsequently passed through a feed-forward network (FFN) to obtain the final fused representations. The following equations summarize this process:

$$\tilde{H}_t^{(N)} = \text{SelfAttn}(H_t^{(N-1)}), \tag{4}$$

$$\hat{H}_t^{(N)} = \text{CrossAttn}(Q = \tilde{H}_t^{(N)}, K = H'_g, V = H'_g), \tag{5}$$

$$H_t^{(N)} = \text{FFN}(\hat{H}_t^{(N)}) \in \mathbb{R}^{L \times d_l}, \tag{6}$$

where $\tilde{H}_t^{(N)}$ denotes the intermediate text representations after self-attention at the N-th decoder layer, $\hat{H}_t^{(N)}$ represents the fused representations obtained via cross-attention, and $H_t^{(N)}$ is the final output after a feed-forward transformation. H'_g corresponds to the graph-aware node embeddings aligned to the LLM embedding space.

3.5 Training

We train the model in an end-to-end manner, jointly optimizing the graph encoder, the graph-text alignment adapter, the cross-attention module, and the LLM. Given an input sequence S consisting of the question concatenated with the textualized subgraph $\mathcal{G}_{sub}$, the model autoregressively predicts the next token. Let the sequence $y = \{y_1, y_2, \ldots, y_T\}$ denote the ground-truth answer. The training objective is to minimize the cross-entropy loss, defined as:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t \mid y_{<t}, S, \theta) \tag{7}$$

where y_t represents the t-th token in the answer sequence. To fine-tune the LLM, we employ LoRA (Low-Rank Adaptation) [5], which adapts the attention layers by adding low-rank updates to the weight matrices while keeping the pre-trained weights frozen.

4 Experiments

4.1 Experiment Setup

Datasets. To validate our proposed method, we conduct extensive experiments on the Graph Question Answering (GraphQA) benchmark [3]. This benchmark comprises three datasets: ExplaGraphs (commonsense reasoning), SceneGraphs (scene graph question answering), and WebQSP (multi-hop knowledge graph question answering). ExplaGraphs and SceneGraphs are evaluated using accuracy, while WebQSP uses the Hits@1 metric. Table 1 presents detailed information on the datasets.

Table 1. Characteristics of datasets

Dataset	#Graph	Task	Metrics
ExplaGraphs	2,766	Commonsense Reasoning	Accuracy
SceneGraphs	100,000	Scene Graph QA	Accuracy
WebQSP	4,737	Knowledge-based QA	Hit@1

Baselines. The compared baselines can be grouped into two categories: LLM-Frozen and LLM-Tuned. For LLM-Frozen methods, we consider six baselines: LLM-only that answers the question directly without graph information, CoT-BAG [19] that appends "Let's construct a graph with the nodes and edges first" after the textual graph, KAPING [1] that enhances inputs with relevant triples retrieved from the graph, Prompt Tuning [8] that introduces learnable soft prompts, GraphToken [13] that encodes graph structure as soft-token sequences, G-Retriever [3] that employs a RAG approach to retrieve relevant subgraphs and encodes subgraphs into graph-level tokens for the LLM. For LLM-Tuned methods, we compare with standard LoRA [5] and G-Retriever with LoRA. For fair comparison, we directly report the performance of all baseline methods from the original G-Retriever paper [3], as they were evaluated on the same dataset split and using identical metrics.

Implementation Details. For the proposed model, we use the open-sourced LLaMA2-7B as the LLM backbone and the Graph Attention Network (GAT) as the GNN backbone. We set the learning rate to 0.0005, batch size to 8, and training epochs to 10. The GNN layers are searched from 2 to 5, and the number of cross-attention heads is tuned from $\{4, 8, 16, 32, 64\}$, with the GNN hidden dimension fixed at 1024. Experiments were conducted on two NVIDIA A100 GPUs with 80GB VRAM.

4.2 Main Results

Table 2 summarizes the experimental results on ExplaGraphs, SceneGraphs, and WebQSP. Among LLM-Frozen methods, simply appending textualized subgraphs (e.g., CoT-BAG) often degrades performance compared to relying on the LLM's internal knowledge alone (LLM-only), highlighting the difficulty of handling noisy or irrelevant graph information without effective fusion. Although prompt tuning with soft prompts provides moderate improvements, it remains limited in capturing the structural semantics of knowledge graphs. In contrast, leveraging a GNN to capture structural features and integrating them into the LLM input space significantly boosts performance (e.g., GraphToken and G-Retriever). In the LLM-tuned methods, fine-tuning the LLM with LoRA further tailors the model to downstream tasks, yielding substantial gains. Our method leverages a cross-attention module at the final decoder layer to dynamically fuse structure-aware node embeddings modeled by GNN with textual context and achieves top-performing results, surpassing the strongest baseline (G-Retriever w/ LoRA) by 6.21%, 0.64%, and 4.21% on ExplaGraphs, SceneGraphs, and WebQSP, respectively. These results validate the effectiveness of structured knowledge integration through our proposed method in enhancing LLM capabilities across diverse reasoning tasks.

Table 2. Overall results on ExplaGraphs, SceneGraphs, and WebQSP across all baselines and our method. Due to the nature of scene graph QA, LLMs cannot answer without graph information (LLM-only). The best results for each dataset are highlighted in bold. $\Delta_{\text{G-Retriever w/ LoRA}}$ denotes the relative improvement of our method over the strongest baseline. Results are reported as mean ± standard deviation.

Method	ExplaGraphs	SceneGraphs	WebQSP
LLM Frozen			
LLM-only	0.6191	-	42.63
CoT-BAG	0.5794	0.5680	39.60
KAPING	0.6227	0.4375	52.64
Prompt tuning	0.5763	0.6341	48.34
GraphToken	0.8508	0.4903	57.05
G-Retriever	0.8516	0.8131	70.49
LLM Tuned			
LoRA	0.8538 (±0.0353)	0.7862 (±0.0031)	66.03 (±0.47)
G-Retriever w/ LoRA	0.8705 (±0.0329)	0.8683 (±0.0072)	73.79 (±0.70)
ours	**0.9246 (±0.0145)**	**0.8739 (±0.0015)**	**76.90 (±1.61)**
$\Delta_{\text{G-Retriever w/ LoRA}}$	↑ 6.21%	↑ 0.64%	↑ 4.21%

4.3 Ablation Study

To assess the contribution of each component, we conduct ablation studies with three variants (Table 3). In w/o the Graph Encoder, initial node embeddings bypass the graph encoder and are directly fed into the Graph-Text Alignment Adapter and the Cross-attention module. In w/o the Alignment Adapter, graph encoder outputs are directly employed for the Cross-attention module. In w/o the Cross-attention, we directly use the pooled node embeddings as the prefix of the input embeddings. Results show that each component contributes differently across datasets. Removing the Graph Encoder degrades performance on all datasets, with a larger drop on WebQSP, likely due to its multi-hop reasoning requiring stronger structural modeling. Excluding the Alignment Adapter impacts ExplaGraphs the most, but only slightly affects SceneGraphs and WebQSP, suggesting semantic alignment is particularly important for commonsense reasoning. Removing Cross-attention leads to significant degradation across all datasets, highlighting its key role in integrating graph information. Overall, the full model achieves the best performance, demonstrating the effectiveness of our method.

4.4 Model Design Comparison

We compare model designs by varying the fusion layer position in the LLM, inserting the cross-attention module at the first, middle (16th), and last (32nd) layers. According to Table 4, fusion at the first layer significantly degrades performance, suggesting that early-stage token representations lack the semantic discriminative power to identify relevant entities, thus introducing irreversible

Table 3. Ablation Study Results

Variant	ExplaGraphs	SceneGraphs	WebQSP
w/o Graph Encoder	0.9093	0.8612	74.10
w/o Alignment Adapter	0.8845	0.8676	76.15
w/o Cross-attention	0.8791	0.8631	73.92
Full Model	**0.9246**	**0.8739**	**76.90**

Table 4. Performance of different fusion layer positions in the LLM.

Fusion Position	ExplaGraphs	SceneGraphs	WebQSP
1 (First)	0.4314	0.2531	13.88
16 (Middle)	0.8953	0.8159	75.36
32 (Last)	0.9246	0.8739	76.90

noise. Middle-layer fusion improves results as contextualized representations develop sufficient coherence to attend to graph nodes, though it occurs before abstractive reasoning is finalized. Last-layer fusion achieves superior performance, likely because: (1) queries encode fully developed reasoning intents after 31 self-attention layers; (2) mature representations effectively filter irrelevant nodes; and (3) fused outputs are directly utilized for answer generation without intermediate dilution. These findings validate that cross-attention effectively fuses high-dimensional embeddings once textual semantics reach sufficient abstraction, even without task-specific pretraining.

4.5 Parameter Sensitivity

Impact of GNN Layers. To investigate the effect of GNN layer depth on model performance, we conduct experiments with GNNs ranging from 2 to 5 layers, with results presented in Figure 2. The findings reveal that the optimal GNN depth varies across datasets with different task types. For ExplaGraphs and SceneGraphs, the best performance is achieved with 4 layers, whereas for WebQSP, 3 layers prove optimal. A common observation is that beyond the optimal depth, increasing the number of layers leads to a performance drop, possibly due to over-smoothing of graph information or model overfitting.

Impact of Cross-Attention Heads. We also evaluate the influence of cross-attention head counts on performance, as shown in Figure 3. The results indicate that the number of attention heads significantly affects model outcomes. For all three datasets, peak performance is observed with 16 heads. Too few heads (e.g., 4 or 8) limit the model's expressive power, while excessive heads (e.g., 32 or 64) may cause attention dilution or computational redundancy, resulting in degraded performance.

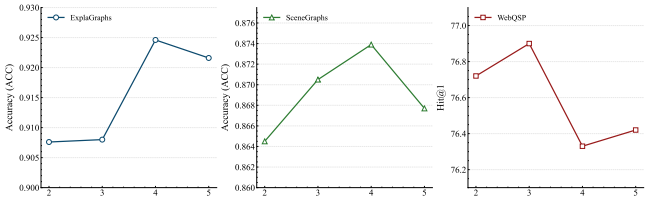

Fig. 2. Performance across different numbers of GNN layers.

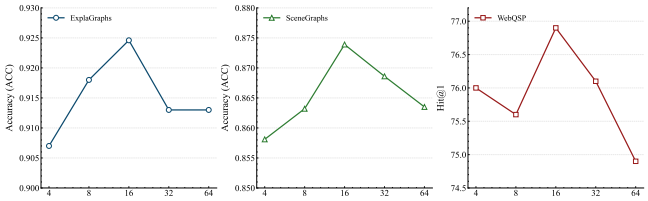

Fig. 3. Performance across different numbers of heads in cross-attention module.

5 Conclusion

In this paper, we propose a more effective graph information fusion method to enhance LLM responses. The method consists of three components: 1) A graph encoder for encoding graph nodes. 2) A graph-text alignment adapter to align semantic spaces, and 3) A cross-attention module in the last layer of the LLM to merge graph information. Extensive experiments on the text-attributed graph datasets demonstrate the effectiveness of our proposed method, with performance improvements of +6.21% on ExplaGraphs, +0.64% on SceneGraphs, and +4.21% on WebQSP.

In the future, we will explore link prediction and graph-text contrastive pretraining to further capture complex structural dependencies and enhance cross-modal alignment, thereby improving model robustness and generalization across diverse tasks.

Acknowledgement.

This work was supported by Key Research and Development Program of Zhejiang Province under Grant 2025C01129 and National Natural Science Foundation of China under Grant 62372145.

References

1. Baek, J., Aji, A.F., Saffari, A.: Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. In: Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations (NLRSE). pp. 78–106 (2023)

2. Chen, Z., et al.: Exploring the potential of large language models (llms) in learning on graphs. SIGKDD Explor. Newsl. **25**(2), 42–61 (2024). https://doi.org/10.1145/3655103.3655110
3. He, X., et al.: G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. In: Advances in Neural Information Processing Systems. pp. 132876–132907 (2024)
4. Houlsby, N., et al.: Parameter-efficient transfer learning for NLP. In: Proceedings of the 36th International Conference on Machine Learning. vol. 97, pp. 2790–2799 (2019)
5. Hu, E.J., et al.: LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (2022), https://openreview.net/forum?id=nZeVKeeFYf9
6. Ji, S., Pan, S., Cambria, E., Marttinen, P., Yu, P.S.: A survey on knowledge graphs: Representation, acquisition, and applications. IEEE Transactions on Neural Networks and Learning Systems **33**(2), 494–514 (2022). https://doi.org/10.1109/TNNLS.2021.3070843
7. Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., Han, J.: Large language models on graphs: A comprehensive survey. IEEE Trans. on Knowl. and Data Eng. pp. 8622–8642 (2024). https://doi.org/10.1109/TKDE.2024.3469578
8. Lester, B., Al-Rfou, R., Constant, N.: The power of scale for parameter-efficient prompt tuning. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing. pp. 3045–3059 (2021). https://doi.org/10.18653/v1/2021.emnlp-main.243
9. Li, X.L., Liang, P.: Prefix-tuning: Optimizing continuous prompts for generation. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL). pp. 4582–4597 (2021). https://doi.org/10.18653/v1/2021.acl-long.353
10. Mahabadi, R.K., Belinkov, Y., Henderson, J.: Variational information bottleneck for effective low-resource fine-tuning. arXiv preprint arXiv:2106.05469 (2021)
11. Minaee, S., et al.: Large language models: A survey. arXiv preprint arXiv:2402.06196 (2025)
12. OpenAI: Gpt-4 technical report. arXiv preprint arXiv:2303.08774 (2024)
13. Perozzi, B., et al.: Let your graph do the talking: Encoding structured data for llms. arXiv preprint arXiv:2402.05862 (2024)
14. Pfeiffer, J., Vulić, I., Gurevych, I., Ruder, S.: MAD-X: An Adapter-Based Framework for Multi-Task Cross-Lingual Transfer. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 7654–7673 (2020). https://doi.org/10.18653/v1/2020.emnlp-main.617
15. Schockaert, S.: Embeddings as epistemic states: Limitations on the use of pooling operators for accumulating knowledge. arXiv preprint arXiv:2210.05723 (2023)
16. Tian, Y., et al.: Graph neural prompting with large language models. Proceedings of the AAAI Conference on Artificial Intelligence pp. 19080–19088 (2024)
17. Touvron, H., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)
18. Vaswani, A., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems (2017)
19. Wang, H., et al.: Can language models solve graph problems in natural language? In: Advances in Neural Information Processing Systems. pp. 30840–30861 (2023)
20. Zhang, X., et al.: GreaseLM: Graph REASoning Enhanced Language Models. In: International Conference on Learning Representations (2022)